

Smart contract audit

rain.interpreter.interface

Content

Project description	3
Executive summary	3
Reviews	3
Technical analysis and findings	4
Security findings	6
**** Critical	6
*** High	6
** Medium	6
M01 - Missing Bit Check in Bloom Filter Lookup	6
* Low	7
L01 - Incomplete Search Space for Bloom Filter Seeds	7
L02 - Zero-Fingerprint Sentinel Collision	7
L03 - Missing MaxDepth Overflow Guard	8
L04 - Missing Structural Validation in lookupWord() Meta Parsing	8
L05 - Missing Index Size Validation in buildParseMetaV2()	9
Informational	9
I01 - Incorrect Mask Constant	9
General Risks and Assumptions	11
Approach and methodology	12



Project description

This repository contains the foundational interfaces and core shared libraries for the Rainlang interpreter ecosystem. It defines the standard specifications for Interpreters, Parsers, Stores, and Expression Deployers across multiple versions (up to V4), ensuring composability and standardised communication between different components of the Rainlang stack.

Executive summary

Type	Library
Languages	Solidity
Methods	Architecture Review, Manual Review, Unit Testing, Functional Testing, Automated Review
Documentation	README.md
Repositories	https://github.com/rainlanguage/rain.interpreter.interface

Reviews

Date	Repository	Commit
13/02/26	rain.interpreter.interface	639b80f929d8a8213fe75e35b446708ba49a8e6a
02/03/26	rain.interpreter.interface	f83d8e0e2ea529acb9b76f622ff73464c65a3ba8

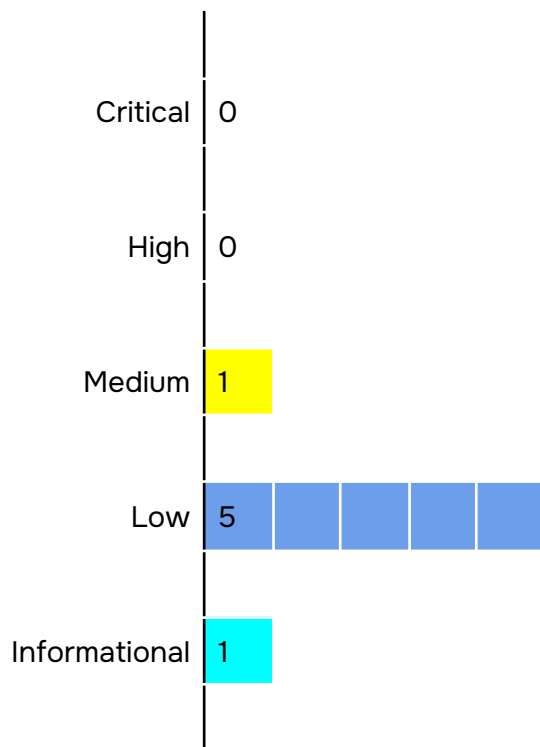
Scope

Contracts
src/error/ErrBytecode.sol
src/error/ErrExtern.sol
src/error/ErrIntegrity.sol
src/interface/IInterpreterCallerV4.sol
src/interface/IInterpreterExternV4.sol



src/interface/IInterpreterStoreV3.sol
src/interface/IInterpreterV4.sol
src/interface/IParserV2.sol
src/interface/ISubParserV4.sol
src/lib/bytecode/LibBytecode.sol
src/lib/caller/LibContext.sol
src/lib/caller/LibEvaluable.sol
src/lib/codegen/LibGenParseMeta.sol
src/lib/ns/LibNamespace.sol
src/lib/parse/LibParseMeta.sol

Technical analysis and findings



Security findings

**** Critical

No critical severity issue found.

*** High

No high severity issue found.

** Medium

M01 - Missing Bit Check in Bloom Filter Lookup

The *lookupWord()* function in *LibParseMeta.sol* contains a logical flaw in how it queries the bloom filter.

When checking if a word exists in a specific bloom filter layer, the code calculates the word's position index (*pos*) based on the population count (*ctpop*) of bits lower than the word's target bit. Crucially, it proceeds to use this *pos* to read a memory slot and compare fingerprints without first verifying if the word's target bit is actually set in the expansion bitmap.

If the bit is 0 (meaning the word is not present in this layer), *pos* will point to the data of a different word (the next set bit). If the 3-byte fingerprint of that unrelated word happens to match the query word's fingerprint (a 1 in ~16.7 million chance), the function returns a false positive match, returning the wrong opcode index.

This breaks the intended 4-byte collision resistance (1 byte from the bloom bit + 3 bytes from the fingerprint) down to just 3 bytes, significantly increasing the risk of parser errors.

Path: *src/lib/parse/LibParseMeta.sol*

Recommendation: Wrap the position calculation and fingerprint check inside a conditional block that ensures the bit is set.

Found in: 639b80f9

Status: Fixed in f83d8e0e



* Low

L01 - Incomplete Search Space for Bloom Filter Seeds

In the `findBestExpander()` function, the loop used to find the optimal seed for the bloom filter has an off-by-one error.

`type(uint8).max` is 255. The condition `seed < 255` means the loop iterates from 0 to 254. The seed 255 (0xFF) is never tested.

While this does not cause a crash or incorrect behavior, it slightly reduces the efficiency of the generated parser by excluding one valid seed from the search space. This could theoretically result in a slightly suboptimal bloom filter (requiring more layers/depth than necessary) if 255 happened to be the best seed.

Path: `src/lib/codegen/LibGenParseMeta.sol`

Recommendation: Update the loop condition to include 255. Since `seed` is a `uint256`, there is no risk of overflow.

Found in: 639b80f9

Status: Fixed in f83d8e0e

L02 - Zero-Fingerprint Sentinel Collision

The parser generation logic (`LibGenParseMeta.sol`) and runtime lookup (`LibParseMeta.sol`) treat a fingerprint of 0x000000 as a sentinel value for "empty slot".

1. Generation: When building the metadata, the code checks if (`posFingerprint != 0`) to determine if a slot is occupied.
2. Runtime: When looking up a word, `posData & FINGERPRINT_MASK` is compared.

If a valid word hashes to a fingerprint of exactly 0 (a ~1 in 16.7 million probability), two issues occur:

1. Overwrite: During generation, the slot is seen as "empty", allowing a subsequent word to overwrite it silently.
2. False Positive: During lookup, a query for a word with fingerprint 0 could match any actual empty slot (which contains 0), returning index 0 incorrectly.

Path: `src/lib/codegen/LibGenParseMeta.sol`



Recommendation: Ensure that a valid word's fingerprint is never 0. This can be done by modifying the `wordBitmapped` function to force the hash to be non-zero (e.g., setting the LSB to 1 if the result is 0).

Found in: 639b80f9

Status: Fixed in f83d8e0e

L03 - Missing MaxDepth Overflow Guard

In the `buildParseMetaV2()` function the loop that builds the bloom filters relies on depth not exceeding `maxDepth`.

If the provided `authoringMeta` cannot be compressed within `maxDepth` layers (e.g., due to many hash collisions), the loop continues incrementing depth. When depth reaches `maxDepth`, the array write `seeds[depth]` triggers a generic Solidity panic (Index Out of Bounds) with no context. This makes debugging difficult for users of the generation library.

Path: `src/lib/codegen/LibGenParseMeta.sol`

Recommendation: Add an explicit check to revert with a descriptive error before the array access.

Found in: 639b80f9

Status: Fixed in f83d8e0e

L04 - Missing Structural Validation in `lookupWord()` Meta Parsing

The `lookupWord()` function in `LibParseMeta.sol` assumes that the `meta` bytes array is well-formed and trusted. It reads the depth value from the first byte and computes offsets for the expansion and item sections without validating that these offsets fit within `meta.length`.

Because the function performs multiple `mload` operations using calculated memory addresses, a malformed or attacker-controlled `meta` (e.g., with an inflated depth) may cause out-of-structure memory reads. In Solidity, `mload` does not enforce bounds checks, so execution will continue using zeroed or unrelated memory.

This can result in incorrect position calculations and potentially false-positive matches, making the parser non-strict if meta is not fully trusted.



Path: src/lib/parse/LibParseMeta.sol

Recommendation: Add minimal structural checks, such as:

- `require(meta.length >= 1);`
- `require(meta.length >= 1 + depth * META_EXPANSION_SIZE);`
- ensure the item data region fits within `meta.length`.

Found in: 639b80f9

Status: Fixed in f83d8e0e

L05 - Missing Index Size Validation in buildParseMetaV2()

The `buildParseMetaV2()` function encodes each item as a 4-byte value composed of:

- 1 byte – opcode index
- 3 bytes – word fingerprint

However, the index (`k`) is not validated or masked to ensure it fits within a single byte before being written: `toWrite = wordFingerprint | (k << 0x18)`.

If `authoringMeta.length > 256` (i.e., `k > 255`), the value of `k << 0x18` will exceed 1 byte and begin setting bits beyond the intended 4-byte item boundary. Because the write operation does not truncate `k` to 8 bits, this may result in corrupted metadata layout and inconsistent parsing behavior.

Path: src/lib/codegen/LibGenParseMeta.sol

Recommendation: Add an explicit bound check before generation or validate per-item.

Found in: 639b80f9

Status: Fixed in f83d8e0e

Informational

I01 - Incorrect Mask Constant

In `LibGenParseMeta.sol`, the constant `META_ITEM_MASK` is calculated incorrectly for its intended use.

`META_ITEM_SIZE` is defined as 4 (bytes). This calculation results in $(1 << 4) - 1 = 15$ (0xF), which is a 4-bit mask. The intention is to mask the full 4-byte (32-bit) item slot.



When used later to clear memory before writing (*and(mload(writeAt), ~META_ITEM_MASK)*), it only clears the last 4 bits instead of the full 32 bits.

This does not cause actual corruption in the current code because the memory is freshly allocated (zero-initialized) and each slot is written only once. However, it is technically incorrect and fragile.

Path: `src/lib/codegen/LibGenParseMeta.sol`

Recommendation: Update the definition to represent a full 32-bit mask.

Found in: 639b80f9

Status: Fixed in f83d8e0e



General Risks and Assumptions

- **Bytecode Integrity Assumption:** LibBytecode is optimized for runtime gas efficiency and deliberately does not perform deep validation of bytecode structure during every read. It operates on the strict assumption that the bytecode headers (source counts, stack allocations, etc.) are accurate. It is the responsibility of the Expression Deployer to perform rigorous integrity checks at deployment time to ensure the headers match the actual bytecode logic. If integrity checks are bypassed or flawed, LibBytecode usage could result in undefined behavior.
- **Context Authentication Responsibility:** While LibContext provides the mechanism to handle signed context data, it does not inherently authorize who can sign. The system assumes that the Calling Contract correctly authenticates signers and that the Expression Logic enforces semantic constraints (such as deadlines, nonces, and domain separators) to prevent replay attacks or misuse of valid signatures.



Approach and methodology

To establish a uniform evaluation, we define the following terminology in accordance with the OWASP Risk Rating

Methodology:

	Likelihood Indicates the probability of a specific vulnerability being discovered and exploited in real-world scenarios
	Impact Measures the technical loss and business repercussions resulting from a successful attack
	Severity Reflects the comprehensive magnitude of the risk, combining both the probability of occurrence (likelihood) and the extent of potential consequences (impact)

Likelihood and impact are divided into three levels: High H, Medium M, and Low L. The severity of a risk is a blend of these two factors, leading to its classification into one of four tiers: Critical, High, Medium, or Low.

When we identify an issue, our approach may include deploying contracts on our private testnet for validation through testing. Where necessary, we might also create a Proof of Concept PoC to demonstrate potential exploitability. In particular, we perform the audit according to the following procedure:

	Advanced DeFi Scrutiny We further review business logics, examine system operations, and place DeFi-related aspects under scrutiny to uncover possible pitfalls and/or bugs
	Semantic Consistency Checks We then manually check the logic of implemented smart contracts and compare with the description in the white paper.
	Security Analysis The process begins with a comprehensive examination of the system to gain a deep understanding of its internal mechanisms, identifying any irregularities and potential weak spots.

